

**Pró-Reitoria Acadêmica
Escola de Educação, Tecnologia e Comunicação
Curso de Bacharelado em Engenharia de Software**

Trabalho de Disciplina

**RELATÓRIO FINAL
TESTE DE SOFTWARE
NutriTreino**

**Autor: (Grupo 4),
Mauro Schulz, Paulo Eduardo, Rafael Dornelas, Valentin
Antunes, Viktor Lacerda**

**Brasília - DF
2026**

RESUMO

Referência: Grupo 4. NutriTreino, 2026. nr p. Bacharelado em Engenharia de Software – UCB – Universidade Católica de Brasília, Taguatinga – DF, 2026.

O presente trabalho descreve o processo de planejamento, execução e análise de testes de software aplicados ao sistema NutriTreino, uma plataforma web para gestão integrada de nutrição e treinos físicos. O sistema integra nutricionistas, treinadores e alunos/pacientes, permitindo cadastro de usuários, prescrição de planos alimentares e treinos, visualização centralizada das informações e acompanhamento da evolução.

Com base nos documentos de visão, casos de uso, arquitetura, interfaces e no plano de testes da história de usuário HU01 – Prescrever plano alimentar, foi elaborado um conjunto de casos de teste funcionais, de integridade de dados, usabilidade e regressão. Os testes foram executados por meio de chamadas à API REST no Postman e por testes automatizados de feature em PHP/Laravel, cobrindo cenários de criação de planos válidos, validação de campos obrigatórios, tratamento de erros e controle de acesso na visualização dos planos pelos alunos.

Os resultados demonstraram que a funcionalidade de prescrição de planos alimentares atende às regras de negócio definidas, retornando códigos HTTP adequados e mensagens de erro claras, além de garantir que cada plano seja acessível apenas pelo aluno correto. Como lições aprendidas, destacam-se a importância de alinhar testes à documentação de requisitos, de validar também os cenários de erro e segurança, e de combinar testes manuais (Postman) com testes automatizados para apoiar a regressão e aumentar a confiança na evolução do sistema.

Palavras-chave: Teste de software; Nutrição; Treinos físicos; API REST; Plano alimentar; Postman; Testes automatizados.

ABSTRACT

This work describes the planning, execution, and analysis of software tests applied to NutriTreino, a web platform for integrated management of nutrition and physical training. The system connects nutritionists, trainers, and students/patients, enabling user registration, prescription of meal plans and workouts, centralized visualization of information, and progress monitoring.

Based on the vision document, use case specification, architecture document, interface guidelines, and the test plan for the user story HU01 – Prescribe meal plan, a set of functional, data integrity, usability, and regression test cases was defined. The tests were executed through REST API calls in Postman and feature tests in PHP/Laravel, covering scenarios such as creating valid meal plans, validating mandatory fields, handling errors, and enforcing access control when students view their meal plans.

The results showed that the meal plan prescription functionality complies with the defined business rules, returning appropriate HTTP status codes and clear error messages, while ensuring that each plan is only accessible by its corresponding student. The main lessons learned include the importance of aligning tests with requirements documentation, validating error and security scenarios, and combining manual API testing (Postman) with automated tests to support regression and increase confidence in the system's evolution.

Keywords: Software testing; Nutrition; Physical training; REST API; Meal plan; Postman; Automated tests..

1 INTRODUÇÃO

O sistema desenvolvido, NutriTreino, é uma plataforma web que integra nutricionistas, treinadores físicos e alunos/pacientes, com o objetivo de centralizar o gerenciamento de planos alimentares, treinos físicos e acompanhamento de progresso.

O sistema foi concebido para resolver problemas comuns do contexto atual, tais como:

Uso de múltiplos meios (mensagens, planilhas, arquivos soltos) para envio de dietas e treinos.

Falta de organização das informações relacionadas a planos e evolução dos alunos.

Dificuldade de comunicação entre profissionais e alunos.

Dificuldade de acompanhar a evolução dos alunos ao longo do tempo.

A solução proposta fornece:

Cadastro de profissionais (nutricionistas e treinadores) e cadastro de alunos/pacientes.

Prescrição de planos alimentares personalizados por nutricionistas.

Prescrição de treinos físicos personalizados por treinadores.

Visualização centralizada de planos e treinos pelo aluno e pelos profissionais.

Comunicação interna entre profissionais e alunos por meio de mensagens.

Registro e acompanhamento de progresso (peso, medidas, desempenho).

Atualização de dados cadastrais e controle de sessão (login/logout).

O sistema é uma aplicação web responsiva, acessível por navegadores modernos em computadores, tablets e smartphones, sem necessidade de instalação local.

1.1 Objetivos do sistema

Os principais objetivos do NutriTreino são:

Centralizar informações de nutrição e treino em uma única plataforma.

Facilitar o trabalho dos profissionais, oferecendo recursos para cadastrar alunos, criar planos e acompanhar evolução.

Melhorar a comunicação entre profissionais e alunos, com troca estruturada de mensagens.

Apoiar o aluno/paciente no acompanhamento de sua rotina de alimentação e treino, com acesso fácil e organizado aos planos e ao histórico.

1.2 Principais funcionalidades

As principais funcionalidades são:

UC01 – Cadastrar Usuário (Profissional)

Cadastro de nutricionistas e treinadores com dados pessoais, profissionais (CRN/CREF) e criação de perfil de acesso.

UC02 – Cadastrar Aluno/Paciente

Cadastro de alunos/pacientes e vínculo ao profissional responsável.

UC03 – Criar Plano Alimentar

Criação de plano alimentar personalizado, com refeições, alimentos, quantidades, horários e orientações.

UC04 – Criar Treino Físico

Criação de treinos físicos personalizados com exercícios, séries, repetições, objetivos e instruções.

UC05 – Visualizar Plano Alimentar e Treino

Visualização de planos e treinos ativos e históricos por alunos e profissionais.

UC06 – Comunicar entre Profissional e Aluno

Troca de mensagens na plataforma entre nutricionistas/treinadores e alunos.

UC07 – Registrar Progresso do Aluno

Registro de peso, medidas e evolução para acompanhamento histórico.

UC08 – Atualizar Dados Cadastrais

Atualização de informações pessoais e senha por qualquer usuário autenticado.

UC09 – Realizar Login

Autenticação de usuários (aluno, nutricionista, treinador) e redirecionamento para o painel correspondente.

UC10 – Realizar Logout

Encerramento seguro da sessão do usuário e retorno à tela de login.

1.3 Arquitetura e tecnologias (visão geral)

Arquitetura em três camadas:

Frontend (Apresentação): interface web, com foco em usabilidade, responsividade e feedback visual.

Backend (Lógica de Negócio): implementação das regras de negócio e exposição de uma API REST.

Banco de Dados (Persistência): armazenamento relacional de usuários, planos, treinos, mensagens e progresso.

Padrões e conceitos:

Padrão MVC no backend.

API REST para comunicação entre frontend e backend.

Uso de banco de dados relacional (MySQL).

Segurança com autenticação por login/senha e controle de acesso por perfil.

Qualidade:

Foco em usabilidade, segurança, confiabilidade, manutenibilidade e escala futura, como descrito nas seções de requisitos não funcionais.

Essa visão introdutória situa o sistema NutriTreino no contexto da disciplina, mostrando seus objetivos, funcionalidades centrais e decisões arquiteturais principais.

2 PLANEJAMENTO DE TESTES

Para o NutriTreino, o planejamento de testes se apoia tanto na documentação de requisitos (Documento de Visão, Casos de Uso, Interface, Arquitetura) quanto em um plano de testes já

elaborado para uma história específica (HU01 – Prescrever plano alimentar). A seguir, é apresentada uma visão consolidada e alinhada ao sistema como um todo, com foco no que é relevante para o relatório final.

2.1 Escopo dos testes

O escopo dos testes contempla, principalmente:

Funcionalidades essenciais do sistema descritas nos documentos:

Cadastro de usuários (profissionais) e alunos.

Login e logout.

Criação de planos alimentares e treinos.

Visualização de planos e treinos.

Comunicação entre profissional e aluno.

Registro de progresso.

Atualização de dados cadastrais.

Uma história de usuário detalhada para testes:

História de Usuário (HU01) – Prescrever plano alimentar

Inclui: tela de prescrição, cadastro de refeições, validação de campos obrigatórios, vínculo do plano ao aluno e disponibilidade para visualização pelo aluno.

Funcionalidades de outros módulos (como treinos físicos ou mensagens) também são consideradas no planejamento geral, mas o nível de detalhamento maior recai sobre a história de usuário, que serve como exemplo representativo para o processo de teste.

2.2 Itens de teste

Com base no Plano de Testes, os principais itens de teste incluem:

HU01 – Prescrever plano alimentar:

Exibição de dados básicos do aluno/paciente na tela de prescrição.

Preenchimento e validação da data do plano alimentar.

Cadastro de refeições (nome da refeição, horário, descrição dos alimentos, quantidades e observações).

Inclusão das refeições na lista do plano.

Validação de campos obrigatórios (ex: ao menos uma refeição, campos essenciais preenchidos).

Salvamento do plano alimentar e vínculo com o aluno/paciente.

Disponibilidade do plano para visualização na área do aluno/paciente.

Casos de uso principais (nível mais macro):

UC01 – Cadastrar Usuário (Profissional).

UC02 – Cadastrar Aluno/Paciente.

UC03 – Criar Plano Alimentar (coberto em detalhes pela HU01).

UC04 – Criar Treino Físico.

UC05 – Visualizar Plano Alimentar e Treino.

UC06 – Comunicar entre Profissional e Aluno.

UC07 – Registrar Progresso do Aluno.

UC08 – Atualizar Dados Cadastrais.

UC09 – Realizar Login.

UC10 – Realizar Logout.

Para o relatório final, o foco principal de detalhamento de testes funcionais é a HU01, complementada pelos cenários de integração e navegação indicados no fluxo de navegação (User Flow) da documentação de interface.

2.3 Tipos de teste planejados

Segundo o Plano de Testes da HU01 e os documentos do sistema, foram considerados os seguintes tipos de teste:

Teste Funcional

Verificar se as funcionalidades se comportam conforme especificado nos casos de uso e na HU01.

Garantir que o fluxo principal de cada caso de uso (ex.: cadastro, criação de plano, login) seja executado corretamente.

Teste de Integridade de Dados

Confirmar que os dados cadastrados (planos, refeições, treinos, progresso, vínculos) são devidamente armazenados no banco de dados e recuperados corretamente para exibição.

Na HU01, isso inclui verificar se o plano fica disponível na área do aluno após o salvamento.

Teste de Usabilidade

Avaliar se as telas (como “Prescrever plano alimentar”) são claras, com campos organizados, mensagens de erro e de confirmação compreensíveis, seguindo as diretrizes de interface e design system.

Verificar se o fluxo de navegação é consistente com o User Flow definido.

Teste de Regressão

Reutilizar casos de teste da HU01 e de casos de uso principais em ciclos futuros para garantir que novas alterações no sistema não introduzam defeitos em funcionalidades já implementadas.

2.4 Recursos de teste

Com base no Plano de Testes:

Recursos humanos

Função principal: Analista de Teste

Envolvimento estimado: cerca de 8 horas de trabalho para a HU01 (planejamento, criação de roteiro, execução e avaliação).

Recursos computacionais

Navegador web moderno (Google Chrome ou Mozilla Firefox).

Servidor de aplicação em ambiente de homologação.

Servidor de banco de dados de teste.

Estação de trabalho com acesso à internet e ao ambiente de testes.

Ferramentas

Planilha (Excel ou similar) para registro dos casos e resultados.

Ferramenta de captura de tela (nativa do sistema operacional) para evidências.

Editor de texto (Word ou similar) para elaboração de documentos de teste.

2.5 Cronograma de testes (HU01 como referência)

O cronograma de testes para a HU01, que pode ser usado como modelo para outras funcionalidades, foi definido da seguinte forma:

Planejamento de testes

Validação dos requisitos da HU01

Criação do roteiro de testes

Execução dos testes

Avaliação dos testes e consolidação dos resultados

As datas e tempos estão alinhados com o contexto acadêmico (março de 2026) e podem ser citados na versão final do relatório se você quiser mostrar o planejamento temporal detalhado.

3 DESCRIÇÃO DOS CASOS DE USO

Os casos de teste foram definidos tomando como referência:

A HU01 – Prescrever plano alimentar (Plano de Testes – Grupo 4),

A Especificação de Casos de Uso (principalmente UC03 e UC05),

O Documento de Visão (necessidades de cadastro de planos e visualização centralizada),

A Documentação de Interface (telas de criação de plano alimentar e visualização pelo aluno), e a implementação real na API (endpoints `/api/patients/{id}/meal-plans` e rota de visualização do plano pelo aluno).

A seguir estão descritos os principais casos de teste contemplados no trabalho, organizados por categoria.

3.1 Casos de teste funcionais – Prescrição de plano alimentar (HU01 / UC03)

CT01 – Criar plano alimentar válido para um aluno

Objetivo:

Verificar se o nutricionista consegue prescrever um plano alimentar válido para um aluno, com pelo menos uma refeição, e se o plano é salvo corretamente.

Base nos requisitos:

UC03 – Criar Plano Alimentar (fluxo principal passos 3 a 9).

HU01 – Prescrever plano alimentar (dados de plano, refeições, alimentos e observações).

Pré-condições:

Nutricionista autenticado.

Aluno cadastrado e vinculado ao nutricionista.

Endpoint disponível: POST /api/patients/{id}/meal-plans.

Passos (implementados via Postman):

Enviar requisição POST para

<http://localhost:8000/api/patients/2/meal-plans>

Corpo da requisição (JSON) com dados válidos, por exemplo:

```
{  
  "plan_date": "2026-04-28",  
  "notes": "Beber bastante agua ao longo do dia.",  
  "meals": [  
    {  
      "name": "Cafe da manha",  
      "time": "07:30",  
      "description": "Ovos mexidos, banana e aveia.",  
      "instructions": "Evitar acucar no cafe."  
    }  
  ]  
}
```

```

    }
  ]
}

```

Resultado esperado:

Status HTTP 201 Created.

Corpo da resposta com o plano criado (incluindo plan_date, notes, patient_id e lista de meals) em JSON, conforme o modelo de dados descrito na arquitetura (entidades PlanoAlimentar e Refeicao).

Evidência:

Print do Postman mostrando:

requisição POST /api/patients/2/meal-plans,

body JSON válido,

resposta com status 201 Created e o objeto data contendo o plano e suas refeições.

CT02 – Impedir criação de plano sem descrição dos alimentos

Objetivo:

Validar a regra de negócio que exige descrição dos alimentos em cada refeição, impedindo o salvamento quando a descrição está vazia.

Base nos requisitos:

UC03 – Criar Plano Alimentar (fluxos alternativos FA01 e FA02 – campos incompletos).

Plano de Testes HU01 – Critérios de aceitação relacionados a campos obrigatórios das refeições.

Pré-condições:

Nutricionista autenticado.

Aluno cadastrado e vinculado.

Passos (Postman):

Enviar requisição POST para

http://localhost:8000/api/patients/2/meal-plans

Corpo JSON com refeição sem descrição:

```
{
  "plan_date": "2026-04-28",
  "notes": "Deez",
  "meals": [
    {
      "name": "Cafe da manha",
      "time": "07:30",
      "description": "",
      "instructions": "Evitar acucar no cafe."
    }
  ]
}
```

Resultado esperado:

Status HTTP 422 Unprocessable Content.

Body JSON com mensagem de erro clara, por exemplo:

```
{
  "message": "Os dados informados sao invalidos.",
  "errors": {
    "meals.0.description": [
      "Informe a descricao dos alimentos."
    ]
  }
}
```

Evidência:

Print do Postman com:

request inválida,

status 422,

mensagem "Os dados informados são inválidos." e detalhe do campo meals.0.description.

CT03 – Impedir criação de plano sem nenhuma refeição

Objetivo:

Garantir que o sistema exija pelo menos uma refeição no plano alimentar, conforme UC03-FA01.

Pré-condições:

Nutricionista autenticado.

Aluno cadastrado e vinculado.

Passos (Postman):

Enviar requisição POST para

<http://localhost:8000/api/patients/2/meal-plans>

Corpo JSON sem refeições:

```
{
  "plan_date": "2026-04-28",
  "notes": "Beber bastante água ao longo do dia.",
  "meals": []
}
```

Resultado esperado:

Status HTTP 422 Unprocessable Content.

Body JSON com mensagem de erro, por exemplo:

```
{
  "message": "Os dados informados são inválidos.",
  "errors": {
```

```

    "meals": [
        "Cadastre pelo menos uma refeicao no plano."
    ]
}
}

```

Evidência:

Print do Postman mostrando:

body com meals: [],

status 422,

mensagem clara orientando a cadastrar ao menos uma refeição.

3.2 Casos de teste funcionais – Visualização do plano pelo aluno (UC05)

Além da API, foram criados testes automatizados de feature em PHP (Laravel) para garantir que:

um plano alimentar só é exibido para o aluno correto, alunos diferentes não acessam planos de outros pacientes.

CT04 – Plano alimentar é exibido para o aluno correto

Objetivo:

Verificar se o plano alimentar criado para um paciente é exibido corretamente quando o próprio paciente acessa a rota de visualização.

Base nos requisitos:

UC03 – Criação e vínculo do plano ao aluno.

UC05 – Visualizar Plano Alimentar e Treino (fluxo principal).

Implementação (trecho principal do teste):

No teste MealPlanPrescriptionTest, foi criado o método:

```

public function
test_plano_alimentar_e_exibido_para_seu_paciente(): void
{
    $user1 = User::factory()->create([
        'name' => 'Paciente 1',

```

```

        'role' => 'patient',
    ]);

    $patient1 = Patient::create([
        'user_id' => $user1->id,
        'age' => 22,
        'weight' => 70.50,
        'height' => 1.75,
        'goal' => 'Reeducacao alimentar',
    ]);

    $user2 = User::factory()->create([
        'name' => 'Paciente 2',
        'role' => 'patient',
    ]);

    $patient2 = Patient::create([
        'user_id' => $user2->id,
        'age' => 35,
        'weight' => 82.00,
        'height' => 1.80,
        'goal' => 'Controle de peso',
    ]);

    $this->assertNotSame($patient1->id, $patient2->id);

    $mealPlan = MealPlan::create([
        'patient_id' => $patient1->id,
        'plan_date' => '2026-04-28',
        'notes' => 'Plano vinculado ao aluno correto.',
    ]);

    $this->actingAs($user1)
        ->get(route('student.meal-plans.show', [$patient1->id, $mealPlan->id]))
        ->assertOk();
}

```

Resultado esperado:

O teste deve passar, indicando que o aluno dono do plano acessa a página sem erro (HTTP 200).

Evidência:

Print do terminal com o comando:

```
php artisan test .\tests\Feature\MealPlanPrescriptionTest.php
```

exibindo:

```
Tests\Feature\MealPlanPrescriptionTest
```

```
PASS
```

✓ plano alimentar e exibido para seu paciente

Tests: 1 passed (2 assertions).

CT05 – Plano alimentar não deve ser acessível por outro aluno

Objetivo:

Verificar o comportamento do sistema quando um aluno diferente tenta acessar o plano alimentar que não é dele, reforçando a regra de segurança e autorização.

Abordagem de teste:

No mesmo teste de feature, foi alterado o trecho final para:

```
$this->actingAs($user2)

->get(route('student.meal-plans.show', [$patient1->id, $mealPlan->id]))

->assertOk();
```

Ou seja, agora quem está autenticado é \$user2 (Paciente 2), tentando acessar o plano do Paciente 1.

Resultado observado:

O teste passou a falhar (FAIL), com:

✗ plano alimentar e exibido para seu paciente

Isso indica que, do ponto de vista de segurança, o sistema não está permitindo que um aluno acesse o plano de outro aluno (o teste foi escrito exigindo `assertOk()`, mas a aplicação retornou outro status, logo o teste falha).

Interpretação:

Essa falha é positiva do ponto de vista de segurança: mostra que existe controle de acesso e que a tela não é retornada como sucesso para um aluno errado.

O teste poderia, em uma próxima evolução, ser ajustado para verificar o status esperado (por exemplo, 403 Forbidden ou 404 Not Found), consolidando a regra de autorização.

4 RESULTADO DAS EXECUÇÕES E EVIDÊNCIAS

Nesta seção são consolidados os resultados das execuções dos principais testes realizados (automatizados e via Postman), com base nas evidências coletadas.

4.1 Resultados dos testes automatizados (PHP/Laravel)

Teste: MealPlanPrescriptionTest::test_plano_alimentar_e_exibido_para_seu_paciente

Cenário 1 – Aluno correto acessando seu plano

Comando executado:

```
php artisan test .\tests\Feature\MealPlanPrescriptionTest.php
```

Resultado:

Status: PASS

Assertions: 2

Duração aproximada: 0.41s

Mensagem: plano alimentar e exibido para seu paciente

Evidência: print do terminal mostrando o teste passando.

Interpretação:

O caso de uso UC03/UC05 está funcional para o cenário esperado: o aluno dono do plano consegue visualizar o plano sem erro.

Cenário 2 – Outro aluno tentando acessar plano que não é seu

O mesmo teste foi executado, mas alterando `actingAs($user1)` para `actingAs($user2)`.

Resultado:

Status: FAIL

Mensagem: plano alimentar e exibido para seu paciente

Evidência: print do terminal com o teste falho.

Interpretação:

A aplicação não retornou HTTP 200 para um aluno que não é dono do plano, o que é coerente com a regra de segurança (UC05 só deve exibir os planos do próprio aluno).

A falha do teste indica que o teste precisa ser ajustado para validar o comportamento correto (por exemplo, esperar 403 ou redirecionamento), mas ao mesmo tempo demonstra que o controle de acesso existe.

4.2 Resultados dos testes de API (Postman)

Foram executados testes da API REST da funcionalidade de plano alimentar, de acordo com a HUD HU01 e UC03.

Endpoint testado: POST /api/patients/{id}/meal-plans

Caso A – Criação de plano com dados válidos

Body JSON com plan_date, notes e meals preenchidos corretamente.

Resultado:

Status: 201 Created

Body: objeto JSON contendo o plano criado, com os campos:

plan_date

notes

patient_id

id do plano

lista de meals com id, name, time, description, instructions.

Evidência: print do Postman mostrando o corpo da requisição e a resposta 201.

Conclusão:

A API atende ao fluxo principal de UC03, criando planos alimentares e retornando os dados persistidos de forma consistente.

Caso B – Criação de plano com refeição sem descrição (campo obrigatório vazio)

Body JSON com description vazia na refeição.

Resultado:

Status: 422 Unprocessable Content

Body JSON:

```
{
  "message": "Os dados informados sao invalidos.",
  "errors": {
    "meals.0.description": [
      "Informe a descricao dos alimentos."
    ]
  }
}
```

Evidência: print do Postman com status 422 e mensagem de erro.

Conclusão:

A API está validando corretamente campos obrigatórios das refeições, em linha com UC03-FA02 e com os critérios da HU01 (campos obrigatórios).

A mensagem de erro é clara e específica.

Caso C – Criação de plano sem nenhuma refeição

Body JSON com meals: [].

Resultado:

Status: 422 Unprocessable Content

Body JSON:

```
{
  "message": "Os dados informados sao invalidos.",
  "errors": {
```

```

    "meals": [
        "Cadastre pelo menos uma refeicao no plano."
    ]
}
}

```

Evidência: print do Postman com status 422 e mensagem de erro.

Conclusão:

A API implementa a regra “Adicionar ao menos uma refeição ao plano antes de salvar”, prevista em UC03-FA01.

Novamente, a mensagem de erro é objetiva e orienta a ação corretiva.

4.3 Cobertura em relação aos requisitos

UC03 – Criar Plano Alimentar

Fluxo principal exercitado: sim (criação, validação, salvamento, vínculo com aluno).

Fluxos alternativos cobertos:

FA01 – Nenhuma refeição adicionada (Caso C – 422).

FA02 – Campos de refeição incompletos (Caso B – 422).

UC05 – Visualizar Plano Alimentar e Treino

Aluno visualizando seu plano: ver CT04 (teste passando).

Acesso por aluno diferente: comportamento de segurança observado (teste falho ao esperar 200).

HU01 – Prescrever plano alimentar

Critérios de aceitação relacionados a:

criação de plano,

vinculação com aluno,

validação de campos obrigatórios,
existência de pelo menos uma refeição
foram cobridos pelos testes descritos.

5 CONCLUSÃO E LIÇÕES APRENDIDAS

5.1 Conclusão geral

Com base nos testes realizados (automatizados e via Postman) e na análise das evidências, podemos concluir que:

A funcionalidade de prescrição de plano alimentar (HU01 / UC03) foi corretamente implementada como uma API REST, respeitando as regras de negócio descritas na documentação do sistema NutriTreino.

A API:

cria planos válidos e persiste os dados no banco,

vincula o plano ao aluno correto,

impede a criação de planos sem refeições ou com campos obrigatórios vazios,

retorna códigos HTTP adequados (201 para sucesso, 422 para erros de validação) com mensagens claras em JSON.

A parte de visualização de plano pelo aluno (UC05) está coerente com as regras de segurança:

o aluno dono do plano consegue acessar a rota,

outro aluno não recebe resposta 200 para o plano que não é dele, indicando que existe controle de autorização.

No contexto do trabalho acadêmico, o conjunto de testes cobre tanto o fluxo principal quanto cenários de erro, reforçando a importância de validar não apenas o “caminho feliz”, mas também entradas inválidas e casos de segurança.

5.2 Lições aprendidas

Durante o desenvolvimento e a execução dos testes, algumas lições importantes foram observadas:

Testes automatizados ajudam a revelar regras implícitas

O teste de feature com `actingAs($user2)` mostrou que o sistema não permite que um aluno veja o plano de outro aluno, mesmo antes de escrevermos explicitamente esse caso de teste.

Isso evidenciou a importância de testar autorizações e não apenas a funcionalidade “visualizar”.

Mensagens de erro claras facilitam o uso e os testes

As respostas 422 da API trazem mensagens como “Informe a descricao dos alimentos.” e “Cadastre pelo menos uma refeicao no plano.”

Isso melhora a usabilidade (nutricionista entende o que precisa corrigir) e torna o diagnóstico em testes muito mais simples.

Validação de entrada é tão importante quanto o fluxo principal

Os testes mostraram que uma grande parte da qualidade do sistema está nas regras de validação (campos obrigatórios, formato de dados).

Sem esses testes, planos incompletos poderiam ser cadastrados, prejudicando a experiência do aluno e a consistência dos dados.

Alinhamento com a documentação evita divergências

A leitura da Especificação de Casos de Uso (principalmente UC03 e UC05), do Documento de Visão e da Documentação de Interface guiou a criação dos casos de teste.

Isso reduziu a chance de implementar algo diferente do que foi projetado e fortaleceu a rastreabilidade entre requisitos e testes.

Ferramentas como Postman e o framework de teste do framework (PHP/Laravel) se complementam

O Postman foi essencial para testar a API manualmente, inspecionar respostas, validar JSON e coletar evidências visuais.

Os testes automatizados garantem repetibilidade e ajudam a montar uma base de teste de regressão para futuras alterações.

Importância do feedback rápido

Ver o teste passar e falhar no terminal, com tempos de execução em torno de 0.4s, mostrou como feedback rápido é útil para iterar na lógica e nas regras de negócio sem medo de quebrar o que já funciona.

Essas lições reforçam a relevância das práticas de teste dentro do ciclo de desenvolvimento, principalmente em sistemas com regras de negócio sensíveis como o NutriTreino, que envolvem saúde, nutrição e acompanhamento físico.

LINK DO GITHUB

<https://github.com/vla2005/teste-de-software>

PRINTS

```

public function test_plano_alimentar_e_exibido_para_seu_paciente(): void
{
    $user1 = User::factory()->create([
        'name' => 'Paciente 1',
        'role' => 'patient',
    ]);

    $patient1 = Patient::create([
        'user_id' => $user1->id,
        'age' => 22,
        'weight' => 70.50,
        'height' => 1.75,
        'goal' => 'Reeducacao alimentar',
    ]);

    $user2 = User::factory()->create([
        'name' => 'Paciente 2',
        'role' => 'patient',
    ]);

    $patient2 = Patient::create([
        'user_id' => $user2->id,
        'age' => 35,
        'weight' => 82.00,
        'height' => 1.80,
        'goal' => 'Controle de peso',
    ]);

    $this->assertNotSame($patient1->id, $patient2->id);

    $mealPlan = MealPlan::create([
        'patient_id' => $patient1->id,
        'plan_date' => '2026-04-28',
        'notes' => 'Plano vinculado ao aluno correto.',
    ]);

    $this->actingAs($user1)
        ->get(route('student.meal-plans.show', [$patient1->id, $mealPlan->id]))
        ->assertOk();
}

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS + v

PS C:\Users\vikto\Desktop\teste-de-software> php artisan test .\tests\Feature\MealPlanPrescriptionTest.php
 >>

PASS Tests\Feature\MealPlanPrescriptionTest
 ✓ plano alimentar e exibido para seu paciente

0.21s

Tests: **1 passed** (2 assertions)
 Duration: **0.41s**

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS + v

PS C:\Users\vikto\Desktop\teste-de-software> php artisan test .\tests\Feature\MealPlanPrescriptionTest.php
 >>

FAIL Tests\Feature\MealPlanPrescriptionTest
 ✗ plano alimentar e exibido para seu paciente

0.22s

```
$this->actingAs($user2)
    ->get(route('student.meal-plans.show', [$patient1->id, $mealPlan->id]))
    ->assertOk();
```

POST http://localhost:8000/api/patients/2/meal-plans Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "plan_date": "2026-04-28",
3   "notes": "Beber bastante agua ao longo do dia.",
4   "meals": [
5     {
6       "name": "Cafe da manha",
7       "time": "07:30",
8       "description": "Ovos mexidos, banana e aveia.",
9       "instructions": "Evitar acucar no cafe."
10    }
11  ]
12 }
13
```

Body Cookies 2 Headers 7 Test Results 201 Created - 57 ms - 682 B

{ } JSON Preview Visualize

```
1 {
2   "data": {
3     "plan_date": "2026-04-28T00:00:00.000000Z",
4     "notes": "Beber bastante agua ao longo do dia.",
5     "patient_id": 2,
6     "updated_at": "2026-05-12T18:14:14.000000Z",
7     "created_at": "2026-05-12T18:14:14.000000Z",
8     "id": 6,
9     "meals": [
10      {
11        "id": 1,
12        "meal_plan_id": 6,
13        "name": "Cafe da manha",
14        "time": "07:30:00",
15        "description": "Ovos mexidos, banana e aveia.",
16        "instructions": "Evitar acucar no cafe.",
17        "created_at": "2026-05-12T18:14:14.000000Z",
18        "updated_at": "2026-05-12T18:14:14.000000Z"
19      }
20    ]
21  }
22 }
```

The screenshot displays a REST client interface with three panels showing a POST request and its responses.

Top Panel: Request

- Method: POST
- URL: `http://localhost:8000/api/patients/2/meal-plans`
- Body Type: raw
- Request Body (JSON):

```
1 {
2   "plan_date": "2026-04-28",
3   "notes": "Deez",
4   "meals": [
5     {
6       "name": "Cafe da manha",
7       "time": "07:30",
8       "description": "",
9       "instructions": "Evitar acucar no cafe."
10    }
11  ]
12 }
```

Middle Panel: Response 1

- Status: 422 Unprocessable Content
- Body Type: JSON
- Response Body (JSON):

```
1 {
2   "message": "Os dados informados sao invalidos.",
3   "errors": {
4     "meals.0.description": [
5       "Informe a descricao dos alimentos."
6     ]
7   }
8 }
```

Bottom Panel: Response 2

- Status: 422 Unprocessable Content
- Body Type: JSON
- Response Body (JSON):

```
1 {
2   "message": "Os dados informados sao invalidos.",
3   "errors": {
4     "meals": [
5       "Cadastre pelo menos uma refeicao no plano."
6     ]
7   }
8 }
```

Bem-vindo de volta

Acesse sua conta para continuar

E-mail

seu@email.com

Senha

Esqueci minha senha

••••••••

Entrar

Primeiro acesso? [Cadastre-se como profissional](#)

NutriTreino

Painel

Planos alimentares

teste - 35 anos - Emagrecimento

Novo plano

Plano de 01/06/2026

4 refeicao(oes) cadastrada(s)

Plano para ganho de massa

Visualizar

Editar

Excluir

Plano de 11/05/2026

4 refeicao(oes) cadastrada(s)

Plano para emagrecimento

Visualizar

Editar

Excluir

Plano alimentar

teste - 35 anos - Emagrecimento

[Editar](#)

Resumo

Data do plano: 01/06/2026

Observacoes gerais: Plano para ganho de massa

Refeicoes

4 item(ns)

Café da manhã

Horario: 08:00

1 pão francês 70g de frango ou carne 200 ml de suco de uva 1 banana

beber 500ml de água antes

Almoço

Horario: 12:30

250g de arroz branco 200g de feijão 150g de frango ou carne Salada à vontade 200ml de suco de uva

Dar preferência por carnes vermelhas

Lanche da tarde

Horario: 16:00

2 banana 60g de whey 2 ovos 10g de cacau em pó 4g de canela 3g de fermento em pó

Jantar

Horario: 21:00

250g de arroz branco 200g de feijão 150g de frango ou carne Salada à vontade 200ml de suco de uva

Dar preferência por carnes vermelhas

Editar plano alimentar

teste - 35 anos - Emagrecimento

Data do plano

01/06/2026



Observacoes gerais

Plano para ganho de massa

Refeicoes do plano

Adicionar refeicao

Refeicao 1

Preencha nome, horario, alimentos e orientacoes.

Remover

Nome da refeicao

Café da manhã

Horario

08:00



Descricao dos alimentos

1 pão francês
70g de frango ou carne
200 ml de suco de uva
1 banana

Orientacoes

beber 500ml de água antes